B3 Clean pandas

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track B: Python for data science 2
B3 Clean pandas 3
B3-L1 Dtypes in pandas 3, nullable and string 3
B3-L2 Vectorise instead of apply 7
B3-L3 Method chaining and Copy-on-Write 12
Track B best-practice checklist 17
Appendix: solutions and answer keys 18

Track B: Python for data science

Write Python and run analyses that are correct, reproducible and easy to review.

B3 Clean pandas

Roadmap: Prerequisite for all months. Dtypes, vectorisation, method chaining and Copy-on-Write in pandas 3.

B3-L1 Dtypes in pandas 3, nullable and string

Concept

Every pandas column has a **dtype**. The wrong dtype gives wrong answers without an error: IDs that lose leading zeros, counts that turn into floats, text that sorts as numbers.

Three facts about pandas 3.0:

- **Text columns default to the `str` dtype.** `pd.Series(["a", None]).dtype` prints `str`, not `object`. Missing values in it are `NaN`. Only strings are allowed, so a stray integer no longer hides in a text column.
- **A missing value turns an integer column into `float64`**, unless you ask for a nullable dtype. `pd.Series([1, None])` is `float64`. `pd.Series([1, None], dtype="Int64")` keeps integers and shows the gap as `<NA>`.
- **Setting a value that does not fit raises.** Assigning `1.5` into an `int64` Series raises `TypeError` instead of silently upcasting the column. (Assigning `NaN` still upcasts to `float64`.)

The capital letter matters. `int64` is NumPy's integer with no missing values. `Int64`, `Float64` and `boolean` are pandas **nullable** dtypes that hold `pd.NA`.

Set dtypes where data enters, not later:

```
df = pd.read_csv(
    path,
    dtype={"respondent_id": "str", "postcode": "str", "household_size": "Int64"},
)
```

Without the `dtype` map, `read_csv` reads a postcode such as `01234` as the number `1234.0` once any row is blank. With it, `"01234"` stays a string.

Check the result with `df.dtypes` right after loading. `df.convert_dtypes()` switches every column to the nullable equivalents in one call, useful for exploration. An explicit map is better in pipelines, because it fails loudly when the source changes.

Why it matters

A panel file stores respondent IDs such as `00731`. One wave has a blank ID row, so `read_csv` infers `float64` and the ID becomes `731.0`. The join to the previous wave, where IDs were read as text, matches nothing. The longitudinal report shows 0% retention for a panel that kept 82% of its members.

`dtype={"respondent_id": "str"}` at load time keeps `00731` as written in every wave.

Worked example: weak code

```
1 df = pd.read_csv("panel_w2.csv")
2 df["respondent_id"] = df["respondent_id"].astype(int).astype(str)
3 df["household_size"] = df["household_size"].fillna(0).astype(int)
```

1. **Line 1.** Dtypes inferred: a blank row turns IDs and counts into float64.
2. **Line 2.** Leading zeros were lost at load time. This cannot bring them back.
3. **Line 3.** Replaces 'unknown' with 0, a real value that biases every mean.

Worked example: strong code

```
 1 import pandas as pd
 2
 3 PANEL_DTYPES: dict[str, str] = {
 1     "respondent_id": "str",
 2     "household_size": "Int64",
 6 }
 7
 8
 9 def read_panel(path: str) -> pd.DataFrame:
10     """Read one panel wave with explicit dtypes."""
 3     return pd.read_csv(path, dtype=PANEL_DTYPES)
```

1. **Line 4.** IDs are labels, not numbers: 00731 stays 00731.
2. **Line 5.** Nullable integer: a gap stays <NA>, the rest stay integers.
3. **Line 11.** Dtypes fixed where data enters. In a pipeline, load the map from config.

Decide dtypes at load time. IDs and codes as str, counts with gaps as Int64.

Recap card

- **Default text dtype in pandas 3.0** str, with NaN for missing values. Only strings are allowed.
- **int64 vs Int64** int64 is NumPy, no missing values. Int64 is the pandas nullable integer that holds pd.NA.
- **Keep leading zeros in IDs** Read the column with dtype str in read_csv.

- **Assign 1.5 into an int64 Series on pandas 3** Raises TypeError instead of silently upcasting.
- **Where to set dtypes** At load time, from a dtype map in config. Check df.dtypes right after.

Practice

Exercise 1. Multiple choice (level 1)

On pandas 3.0, what is the dtype of `pd.Series([4, None, 2])`?

- a) int64
- b) Int64
- c) float64
- d) object

Why?

- r1) NumPy int64 cannot hold a missing value, so pandas stores NaN in a float column.
- r2) pandas 3 stores every number as float.

Exercise 2. Predict the output (level 2)

What does this print on pandas 3.0?

What you need to know.

`io.StringIO(text)` wraps a string in a file-like object. `pd.read_csv` reads it as if it were a file on disk, which keeps small examples and tests free of real files.

```
import io

import pandas as pd

pd.read_csv(io.StringIO("a,b\n1,x\n"))   # one row: a=1, b='x'
```

`df["col"].iloc[0]` returns the first value of a column by position.

```
import io

import pandas as pd

csv = "id,postcode\n1,01234\n2,\n"
df = pd.read_csv(io.StringIO(csv))
print(df["postcode"].iloc[0])
```

Exercise 3. Fill the gap (level 2)

Keep IDs as text and let the count column hold missing values as integers.

```
df = pd.read_csv(path, dtype={"respondent_id": "_____(1)",
    "household_size": "_____(2)"})
```

Exercise 4. Write the function (level 3)

Write `read_panel(text: str) -> pd.DataFrame`. It reads CSV text with columns `respondent_id` and `household_size`. Keep `respondent_id` as text with leading zeros, and `household_size` as a nullable integer where blanks become `<NA>`. Take the dtype map from a module constant `PANEL_DTYPES`.

Your code must pass these checks:

- respondent_id must keep its leading zeros. Read it as str.
- household_size must be Int64, with the blank as <NA>.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Module constant.
PANEL_DTYPES = ...

# Read panel CSV text with explicit dtypes.
def read_panel(text: str) -> pd.DataFrame: ...
```

Starter code:

```
import io

import pandas as pd


def read_panel(text):
    return pd.read_csv(io.StringIO(text))
```

Exercise 5. Spot the bug (level 4)

The mean household size reported for wave 3 dropped from 2.6 to 2.1. Nothing changed in the sample. Tap the line that caused it.

```
import pandas as pd


def mean_household(df: pd.DataFrame) -> float:
    """Mean household size among respondents who answered."""
    sizes = df["household_size"].astype("Int64")
    sizes = sizes.fillna(0)
    return float(sizes.mean())
```

Write down the line number of the bug. Then write the fix.

Sources

- In pandas 3.0 text columns default to the str dtype, with NaN as the missing value. (verified: pandas 3.0 release notes; run on pandas 3.0.6 and 3.0.2 in this repository.)
- In pandas 3.0 setting an incompatible value such as 1.5 into an int64 Series raises TypeError. (verified: pandas 3.0 release notes (setitem upcasting removed); run on pandas 3.0.6 and 3.0.2 in this repository.)
- read_csv infers float64 for an integer-like column with a blank value. (verified: Run on pandas 3.0.6 and 3.0.2 in this repository.)

B3-L2 Vectorise instead of apply

Concept

pandas stores each column as an array. A **vectorised** operation, such as `df["a"] * df["b"]`, `np.where()` or `pd.cut()`, runs one compiled loop over the whole array. `df.apply(func, axis=1)` does the opposite. It builds a Series for each row and calls your Python function once per row.

The gap is large. On one million rows, a row-wise `apply` that picks a weight when a flag is set took 4.0 seconds. `np.where` did the same job in 7.5 milliseconds, about 500 times faster. (pandas 3.0.6, NumPy 2.4.6, one laptop core.)

Speed is not the only gain. Vectorised code states the rule for the whole column in one line, so a reviewer checks one expression instead of a branching function.

Reach for these first:

Task	Vectorised tool
Arithmetic between columns	`+`, `-`, `*`, `/` on Series
Two-way choice	`np.where(cond, a, b)`
Many-way choice	`np.select(conditions, choices, default)`
Numeric bands	`pd.cut(values, bins, labels, right=False)`
Lookup from a dict	`Series.map(mapping)`
Limits	`Series.clip(lower, upper)`

Watch missing values. `NaN > 60` is `False`, so `np.where(minutes > 60, "heavy", "light")` labels a missing value `"light"`. Handle missing values on purpose:

`pd.cut` keeps `NaN` as missing, and `Series.where(values.notna())` puts it back after `np.where`.

Keep `apply` for work with no vectorised form, such as calling an external parser on each string. Even then, try `Series.map` or a list comprehension on one column before `apply(axis=1)`.

Why it matters

A weighting script derives 25 columns with `apply(axis=1)`. It ran in 8 minutes on 2 million rows, so nobody noticed. The programme then merged three trackers into one file of 40 million rows. At the measured rate of about 4 seconds per million rows per column, the derivations alone need roughly 67 minutes. The job hit its 60-minute timeout every night for a week, and weighted results for all three trackers were late.

The same script also labelled respondents with a missing `minutes` value as `"light"` viewers, because the row function used `if row["minutes"] > 60`. The vectorised rewrite fixed the speed, and review of the one-line rule exposed the missing-value bug.

Worked example: weak code

```
  1 import pandas as pd
  2
  3
  1 def derive(df):
  5     def per_row(row):
  6         weighted_reach = row["final_weight"] if row["reach_flag"] == 1
    else 0.0
  2         if row["age"] < 35:
  8             band = "16-34"
  9         elif row["age"] < 55:
 10             band = "35-54"
 11         else:
 12             band = "55+"
 13         return pd.Series({"weighted_reach": weighted_reach, "age_group":
    band})
 14
  4     df[["weighted_reach", "age_group"]] = df.apply(per_row, axis=1)
 16     return df
```

1. **Line 4.** No type hints and no docstring. The reader cannot tell what the frame must contain.
2. **Line 7.** `NaN < 35` is False, so a missing age falls through to `55+` without a warning.
3. **Line 15.** Builds a Series and calls Python for every row. About 500 times slower than a vectorised rule.

4. **Line 15.** Mutates the caller's frame. A pure function returns a new frame.

Worked example: strong code

```
 1 import numpy as np
 2 import pandas as pd
 3
 1 AGE_BINS = [16, 35, 55, np.inf]
 5 AGE_LABELS = ["16-34", "35-54", "55+"]
 6
 7
 8 def derive(df: pd.DataFrame) -> pd.DataFrame:
 9     """Add weighted reach and age group.
10
11     Args:
12         df: Respondents with reach_flag, final_weight and age.
13
14     Returns:
15         A new frame. Missing ages give a missing age_group.
16     """
 2     return df.assign(
 3         weighted_reach=np.where(df["reach_flag"] == 1,
   df["final_weight"], 0.0),
 4         age_group=pd.cut(df["age"], bins=AGE_BINS, labels=AGE_LABELS,
   right=False),
20     )
```

1. **Line 4.** Band edges live in one constant, so a change to the bands is a one-line change.
2. **Line 17.** `assign` returns a new frame, so the caller's data is untouched.
3. **Line 18.** One compiled pass over the column.
4. **Line 19.** `right=False` makes each band include its lower edge: 35 goes to `35-54`. Missing ages stay missing.

The strong version is shorter, about 500 times faster on large frames, and handles missing ages on purpose.

Recap card

- **Why is df.apply(func, axis=1) slow?** It calls a Python function once per row. np.where measured about 500 times faster on one million rows.
- **Vectorised if-else in NumPy** np.where(cond, value_if_true, value_if_false). For several conditions, np.select.
- **What does NaN > 60 return?** False. np.where and np.select send missing values to a real category unless you handle them.
- **Numeric bands that keep missing values missing** pd.cut(values, bins, labels, right=False).

Practice

Exercise 1. Multiple choice (level 1)

Which line computes `weighted_reach` for a frame with one million rows in a single compiled pass?

- a) `df.apply(lambda r: r["final_weight"] if r["reach_flag"] == 1 else 0.0, axis=1)`
- b) `[w if f == 1 else 0.0 for w, f in zip(df["final_weight"], df["reach_flag"])]`
- c) `np.where(df["reach_flag"] == 1, df["final_weight"], 0.0)`
- d) `for i, row in df.iterrows(): ...`

Why?

- r1) It evaluates the condition and the choice on whole arrays in compiled code.
- r2) It uses fewer characters than the other options.
- r3) NumPy runs the lambda on several cores.

Exercise 2. Predict the output (level 2)

What does this print?

What you need to know.

`series.tolist()` returns the values of a pandas Series as a plain Python list, so `print` shows them without the index:

```
pd.Series([3, 4]).tolist()   # [3, 4]
```

```
import pandas as pd

s = pd.Series([0.0, 1.5, None])
print((s > 1).tolist())
```

Exercise 3. Fill the gap (level 2)

Complete the line so rows with `reach_flag == 1` keep their weight and every other row gets zero.

```
df["weighted_reach"] = np._____(1)(df["reach_flag"] == 1,
    df["final_weight"], _____(2))
```

Exercise 4. Write the function (level 3)

Write `weighted_reach_pct(df)`. It returns the weighted share of rows with `reach_flag == 1`: the sum of `final_weight` where the flag is 1, divided by the

sum of all `final_weight`. Return `None` when the total weight is zero or the frame is empty. Do not use `apply` or a loop over rows.

Your code must pass these checks:

- For weights 2, 1, 1 with flags 1, 0, 1 the answer is 3 / 4 = 0.75. Divide the flagged weight by the total weight.
- An empty frame must return None, not raise or return NaN. Guard the division.
- When the total weight is zero, return None instead of dividing by zero.
- Return a Python float (wrap the result in `float()`), not a NumPy scalar.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Weighted share of respondents with reach_flag == 1.
def weighted_reach_pct(df: pd.DataFrame) -> float | None: ...
```

Starter code:

```
import numpy as np
import pandas as pd


def weighted_reach_pct(df: pd.DataFrame) -> float | None:
    ...
```

Exercise 5. Refactor (level 4)

Refactor `add_age_group` to vectorised code. Keep the bands: under 35 is `16-34`, under 55 is `35-54`, anything else is `55+`. A missing age must give a missing age group, not `55+`. Do not change the input frame.

Your code must pass these checks:

- The bands are wrong at an edge. 35 belongs to 35-54 and 55 belongs to 55+. With pd.cut, set right=False.
- A missing age must give a missing age group. `np.where` and `np.select` turn NaN into the default band. `pd.cut` keeps it missing.
- Your function changed the caller's frame. Return a new frame with `df.assign(...)`.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Module constant.
AGE_BINS = ...

# Module constant.
AGE_LABELS = ...
```

```python

# Return a copy of df with an age_group column. Missing ages stay missing.
def add_age_group(df: pd.DataFrame) -> pd.DataFrame: ...
```

Starter code:

```python
import pandas as pd


def add_age_group(df):
    def band(row):
        if row["age"] < 35:
            return "16-34"
        if row["age"] < 55:
            return "35-54"
        return "55+"

    df["age_group"] = df.apply(band, axis=1)
    return df
```

Exercise 6. Spot the bug (level 5)

The docstring promises that missing minutes stay missing. A respondent with no minutes recorded shows as `"light"`. Tap the line with the bug, then fix the function.

```python
def add_viewing_band(df: pd.DataFrame) -> pd.DataFrame:
    """Band weekly minutes into heavy (60 or more) and light. Missing minutes stay missing."""
    minutes = df["minutes"]
    band = np.where(minutes >= 60, "heavy", "light")
    return df.assign(viewing_band=band)
```

Write down the line number of the bug. Then write the fix.

Sources

- Row-wise apply was about 500 times slower than np.where on one million rows. (verified: Measured in this repo's build environment: pandas 3.0.6, NumPy 2.4.6, 4.03 s against 7.5 ms.)
- pd.cut keeps NaN as missing and supports right=False. (verified: Run in pandas 3.0.6.)

B3-L3 Method chaining and Copy-on-Write

Concept

Copy-on-Write (CoW) is always on in pandas 3.0. Every indexing result, such as `df[cols]`, `df[mask]` or `df["a"]`, behaves as a separate object.

Changing it never changes `df`. pandas copies data only when you write to a shared object, so reads stay cheap.

One consequence: **chained assignment never works**.

```
df[df["age"] < 18]["weight"] = 0       # df unchanged, ChainedAssignmentError warning
df.loc[df["age"] < 18, "weight"] = 0   # one step: df changed
```

The first line writes to a temporary copy, which Python then discards. pandas 3 emits a `ChainedAssignmentError` warning, but the code keeps running. `SettingWithCopyWarning` no longer exists.

CoW rewards a second habit: **method chaining**. Build the result as one expression of steps that each return a new frame.

```
result = (
    raw
    .rename(columns=COLUMN_MAP)
    .loc[lambda d: d["age"] >= 18]
    .assign(weight=lambda d: d["weight"].clip(upper=5.0))
    .pipe(add_age_band, bins=BINS, labels=LABELS)
)
```

- `assign(col=lambda d: ...)` refers to the frame at that step, not the original `raw`.
- `.loc[lambda d: ...]` filters on the current step.
- `.pipe(func, **kwargs)` calls your own pure function (lesson B2-L1) inside the chain.

Chains read top to bottom, leave no half-finished variables (`df2`, `df_tmp`, `df_final`) in the namespace, and do not mutate the input. Keep each chain to about ten steps. Split longer ones into named functions and join them with `pipe`.

To debug a chain, comment out steps from the bottom up, or add `.pipe(lambda d: print(d.shape) or d)` between steps.

Why it matters

An analyst caps extreme weights with `df[df["weight"] > 5]["weight"] = 5`. On pandas 2 it printed a SettingWithCopyWarning that everyone had learned to ignore. The weights were never capped. Three respondents with weights above 12 moved the reported brand share by 4 points. After the upgrade to pandas 3 the assignment still does nothing, with a different warning.

`df.assign(weight=df["weight"].clip(upper=5.0))` or a single `df.loc[mask, "weight"] = 5` does what the analyst meant.

Worked example: weak code

```
1  df = raw.rename(columns=COLUMN_MAP)
2  df2 = df[df["age"] >= 18]
3  df2[df2["weight"] > 5]["weight"] = 5.0
4  df3 = add_age_band(df2, BINS, LABELS)
5  result = df3
```

1. **Line 2.** Intermediate names that outlive their use and invite edits to the wrong one.
2. **Line 3.** Chained assignment: writes to a temporary copy. The weights are not capped.
3. **Line 5.** Which of df, df2, df3 is the final state? The reader has to trace every line.

Worked example: strong code

```
1  import pandas as pd
2
3
4  def prepare(raw: pd.DataFrame, column_map: dict[str, str], bins:
       list[int], labels: list[str]) -> pd.DataFrame:
5      """Rename, keep adults, cap weights at 5 and add age bands."""
6      return (
7          raw
8          .rename(columns=column_map)
9          .loc[lambda d: d["age"] >= 18]
10         .assign(weight=lambda d: d["weight"].clip(upper=5.0))
11         .pipe(add_age_band, bins=bins, labels=labels)
12     )
```

1. **Line 9.** Filters the renamed frame at this step.
2. **Line 10.** Returns a new frame with capped weights. No assignment to a copy.
3. **Line 11.** Your own pure function, called as one step of the chain.

Under Copy-on-Write, write with .loc in one step, or better, build a new frame with a chain of assign, loc and pipe.

Recap card

- **Copy-on-Write in pandas 3.0** Always on. Indexing results behave as separate objects, copied only on write.
- **df[mask]["col"] = value on pandas 3** Changes nothing and emits ChainedAssignmentError. Use df.loc[mask, "col"] = value.
- **Refer to the current step inside a chain** A lambda: assign(x=lambda d: ...) or loc[lambda d: ...].
- **Call your own function inside a chain** .pipe(func, **kwargs).

Practice

Exercise 1. Multiple choice (level 1)

On pandas 3.0, `df[df["age"] < 18]["weight"] = 0` runs. What happens to `df`?

- a) Weights of under-18s become 0.
- b) Nothing. pandas emits a ChainedAssignmentError warning.
- c) It raises an exception and stops.
- d) Every weight becomes 0.

Why?

- r1) The first indexing step returns a new object, so the write lands on a temporary copy.
- r2) pandas 3 makes every DataFrame read-only.

Exercise 2. Predict the output (level 2)

What does this print on pandas 3.0?

```python
import pandas as pd

df = pd.DataFrame({"a": [1, 2, 3]})
s = df["a"]
s.iloc[0] = 100
print(df["a"].tolist())
```

Exercise 3. Put in order (level 2)

Put the steps of the chain in order. The raw columns are `wt` and `age`. Weights must average 1.0 across the adults in the output.

- .pipe(add_age_band, bins=BINS, labels=LABELS)
- .assign(weight=lambda d: d["weight"] / d["weight"].mean())
- .rename(columns={"wt": "weight"})
- .loc[lambda d: d["age"] >= 18]

Exercise 4. Refactor (level 3)

Rewrite `cap_weights` as one method chain that returns a new frame. Keep only rows where `age >= 18`, then cap `weight` at `cap`. Leave the input unchanged. No chained assignment.

Your code must pass these checks:

- Weights above cap must become cap, among adults only.
- Only rows with age 18 or over must remain.
- The input frame changed.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Keep adults and cap their weights at cap.
def cap_weights(
    df: pd.DataFrame,
    cap: float,
) -> pd.DataFrame: ...
```

Starter code:

```python
def cap_weights(df, cap):
    adults = df[df["age"] >= 18]
    adults[adults["weight"] > cap]["weight"] = cap
    return adults
```

Exercise 5. Spot the bug (level 4)

Weights for invalid interviews should be zero. The check after the function shows they are unchanged. Tap the line to fix, and rewrite the function.

```python
import pandas as pd


def zero_invalid(df: pd.DataFrame) -> pd.DataFrame:
    """Set weight to 0 where valid is False."""
    out = df.copy()
    out[~out["valid"]]["weight"] = 0.0
    return out
```

Write down the line number of the bug. Then write the fix.

Sources

- Copy-on-Write is always enabled in pandas 3.0 and the mode.copy_on_write option has no effect. (verified: pandas 3.0 release notes; Pandas4Warning text observed on pandas 3.0.6 and 3.0.2 in this repository.)
- Chained assignment never updates the original in pandas 3.0 and emits ChainedAssignmentError as a warning. (verified: pandas Copy-on-Write user guide; run on pandas 3.0.6 and 3.0.2 in this repository.)
- SettingWithCopyWarning no longer exists in pandas 3.0. (verified: pandas 3.0 release notes; pandas.errors has no SettingWithCopyWarning on 3.0.6.)

Track B best-practice checklist

☐ The project has a pyproject.toml, a virtual environment and pinned dependencies.

☐ ruff and mypy pass before every commit.

☐ Functions are pure where possible. Side effects sit at the edges.

☐ Dtypes are explicit. pandas code is vectorised, with no row-wise apply on large frames.

☐ Inputs are validated with pandera or pydantic at every boundary.

☐ Seeds, config and data versions are recorded for every result.

☐ Weighted estimates use the survey weights. Intervals and effect sizes go with every estimate.

☐ Preprocessing sits inside a scikit-learn Pipeline, fitted on training folds only.

☐ The metric matches the decision. Calibration and subgroup performance are checked.

☐ Every experiment is tracked in MLflow.

☐ Tests cover edge cases, including empty inputs and zeros.

☐ Tests use fixtures and parametrize, compare numbers with stated tolerances, isolate I/O, and run with coverage in CI.

☐ Every model has a model card. Every decision has a log entry.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

B3-L1 Dtypes in pandas 3, nullable and string

B3-L1 exercise 1

Hints. (1) Can NumPy's int64 hold NaN? (2) What does pandas use for NaN by default? (3) A float.

Answer. c) float64 Because: r1) NumPy int64 cannot hold a missing value, so pandas stores NaN in a float column.

NaN is a float, so a missing value upcasts integers to `float64`. Ask for `Int64` to keep integers.

Common wrong answers.

- `a`: int64 has no way to represent a missing value.
- `b`: Int64 is used only when you ask for it, for example with dtype="Int64" or convert_dtypes().
- `d`: Numbers with None infer a numeric dtype, not object.
- `c|r2`: Right answer, wrong reason. pd.Series([4, 2]) is int64. The None forces the float.

B3-L1 exercise 2

Hints. (1) What dtype does read_csv infer for 01234? (2) What does the blank row add? (3) A number with a missing value is float64.

Answer.

```
1 1234.0
```

`read_csv` reads `01234` as a number and the blank as NaN, so the column is `float64` and the leading zero is gone. Pass `dtype={"postcode": "str"}`.

Common wrong answers.

- `01234`: Without a dtype map, read_csv infers a number. The blank row makes it float64.
- `1234`: The blank row forces float64, so the value prints with .0.

B3-L1 exercise 3

Hints. (1) IDs are labels, not quantities. (2) Which integer dtype holds pd.NA? (3) `str` and `Int64`.

Answer.

```
df = pd.read_csv(path, dtype={"respondent_id": "str", "household_size":
    "Int64"})
```

`str` keeps IDs as written. `Int64` is the nullable integer dtype.

Common wrong answers.

- g2 = `int64`: Lower-case int64 is NumPy's integer and fails on a blank value. Use capital-I Int64.
- g1 = `int`: IDs are labels. Reading them as numbers drops leading zeros.

B3-L1 exercise 4

Hints. (1) read_csv takes a dtype mapping. (2) IDs as str, counts as Int64. (3) `pd.read_csv(io.StringIO(text), dtype=PANEL_DTYPES)`.

Answer.

```
import io

import pandas as pd

PANEL_DTYPES: dict[str, str] = {"respondent_id": "str", "household_size":
    "Int64"}


def read_panel(text: str) -> pd.DataFrame:
    """Read panel CSV text with explicit dtypes.

    Args:
        text: CSV content with respondent_id and household_size.

    Returns:
        A frame with str IDs and nullable integer counts.
    """
    return pd.read_csv(io.StringIO(text), dtype=PANEL_DTYPES)
```

The dtype map fixes both columns at load time. In a real pipeline it comes from config, not code.

B3-L1 exercise 5

Hints. (1) What does mean() do with <NA>? (2) What does it do with 0? (3) Filling blanks with 0 adds fake answers.

Answer. The bug is on line 7.

Fixed version:

```
1 import pandas as pd
2
3
4 def mean_household(df: pd.DataFrame) -> float:
5     """Mean household size among respondents who answered."""
6     sizes = df["household_size"].astype("Int64")
7     return float(sizes.mean())
```

`mean()` skips `<NA>` by default. Filling with 0 turns non-answers into households of size zero and pulls the mean down.

Common wrong answers.

- Line 6: Int64 is right: it keeps blanks as <NA>, which mean() skips. The problem is what happens next.
- Line 8: mean() is fine on its own. It skips <NA>, but by then the blanks are zeros.

B3-L2 Vectorise instead of apply

B3-L2 exercise 1

Hints. (1) Look for the option with no per-row Python code. (2) A vectorised call takes whole columns as arguments. (3) `np.where(cond, a, b)` picks from `a` or `b` element by element in compiled code.

Answer. c) `np.where(df["reach_flag"] == 1, df["final_weight"], 0.0)` Because: r1) It evaluates the condition and the choice on whole arrays in compiled code.

`np.where` takes whole arrays and runs one compiled loop. The other three options call Python code once per row.

Common wrong answers.

- `a`: `apply(axis=1)` calls the lambda once per row in Python. It measured about 500 times slower than `np.where` on one million rows.
- `b`: A list comprehension beats `apply`, but it still runs a Python loop over every value.
- `d`: `iterrows()` builds a Series for each row. It is the slowest option here.
- `c|r2`: Right choice, wrong reason. Length has nothing to do with speed. `np.where` works on whole arrays in compiled code.
- `c|r3`: Right choice, wrong reason. There is no lambda, and `np.where` runs on one core. It is fast because it avoids a Python call per row.

B3-L2 exercise 2

Hints. (1) `None` in a float Series becomes `NaN`. (2) What does any comparison with `NaN` return? (3) Every comparison with `NaN` is `False`.

Answer.

```
[False, True, False]
```

`None` becomes `NaN` in a float64 Series, and `NaN > 1` is `False`. So the missing value is treated as "not above 1". This is the missing-value trap from the concept.

Common wrong answers.

- `[False, True, None]`: A comparison on a float Series returns plain booleans. `NaN > 1` evaluates to `False`, not to a missing value.
- `[False, True, nan]`: Comparisons do not propagate `NaN` on float64 data. They return `False`, which is why missing values get silently mislabelled.
- `[True, True, False]`: `0.0 > 1` is `False`. Only 1.5 exceeds 1.

B3-L2 exercise 3

Hints. (1) The function takes a condition, a value for True and a value for False. (2) Its name is the SQL keyword for filtering rows. (3) `np.where(cond, value_if_true, value_if_false)`, with zero as the false value.

Answer.

```
df["weighted_reach"] = np.where(df["reach_flag"] == 1,
    df["final_weight"], 0.0)
```

`np.where(cond, df["final_weight"], 0.0)` keeps the weight where the flag is 1 and returns 0.0 elsewhere, as one float array.

Common wrong answers.

- g1 = `select`: `np.select` takes a list of conditions and a list of choices. For one condition and two outcomes, use `np.where`.
- g1 = `if`: There is no `np.if`. The vectorised if-else in NumPy is `np.where`.
- g2 = `None`: `None` turns the result into an object array. Use `0.0` so the column stays numeric.

B3-L2 exercise 4

Hints. (1) Compute the total weight first, and return early when it is zero. (2) `df['final_weight'].where(df['reach_flag'] == 1, 0.0)` keeps weights for reached rows only. (3) Sum the masked weights, divide by the total, and wrap the result in `float()`.

Answer.

```python
import numpy as np
import pandas as pd


def weighted_reach_pct(df: pd.DataFrame) -> float | None:
    """Weighted share of respondents with reach_flag == 1.

    Args:
        df: Rows with reach_flag and final_weight.

    Returns:
        The weighted share, or None when the total weight is zero.
    """
    total = df["final_weight"].sum()
    if total == 0:
        return None
    reached = df["final_weight"].where(df["reach_flag"] == 1, 0.0).sum()
    return float(reached / total)
```

Sum the weights once for the denominator and return `None` when it is zero, which covers the empty frame too. Mask weights by the flag, sum, divide and convert to `float`.

B3-L2 exercise 5

Hints. (1) `pd.cut` turns numbers into labelled bands and keeps missing values missing. (2) Bins `[-np.inf, 35, 55, np.inf]` with `right=False` give [.., 35), [35, 55), [55, ..). (3) Wrap it in `df.assign(age_group=pd.cut(...))` so you return a new frame.

Answer.

```python
import numpy as np
import pandas as pd

AGE_BINS = [-np.inf, 35, 55, np.inf]
AGE_LABELS = ["16-34", "35-54", "55+"]


def add_age_group(df: pd.DataFrame) -> pd.DataFrame:
    """Return a copy of df with an age_group column. Missing ages stay missing."""
    return df.assign(
        age_group=pd.cut(df["age"], bins=AGE_BINS, labels=AGE_LABELS, right=False)
    )
```

`pd.cut` with left-closed bins (`right=False`) matches the original `<` comparisons exactly. It keeps `NaN` as missing, which fixes a bug the original had. `assign` returns a new frame.

B3-L2 exercise 6

Hints. (1) What is `NaN >= 60`? (2) `np.where` has only two outcomes, so a missing value gets one of them. (3) Build the band as a Series, then keep it only where `minutes.notna()`.

Answer. The bug is on line 4.

Fixed version:

```
1 def add_viewing_band(df: pd.DataFrame) -> pd.DataFrame:
2     """Band weekly minutes into heavy (60 or more) and light. Missing
    minutes stay missing."""
3     minutes = df["minutes"]
4     band = pd.Series(np.where(minutes >= 60, "heavy", "light"),
    index=df.index)
5     return df.assign(viewing_band=band.where(minutes.notna()))
```

`NaN >= 60` is `False`, so `np.where` returns `"light"` for missing minutes. Wrap the result in a Series on the same index and apply `.where(minutes.notna())` to restore the missing values.

Common wrong answers.

- Line 3: Line 3 only selects the column. The missing value is lost where the condition is evaluated.
- Line 5: `assign` is right. It writes whatever `band` holds, and `band` is already wrong.

B3-L3 Method chaining and Copy-on-Write

B3-L3 exercise 1

Hints. (1) What does df[mask] return under Copy-on-Write? (2) Where does the second step write? (3) To a copy that is thrown away.

Answer. b) Nothing. pandas emits a ChainedAssignmentError warning. Because: r1) The first indexing step returns a new object, so the write lands on a temporary copy.

Two indexing steps mean two objects. The write hits the temporary one. Write in one step with `.loc`, or use `assign`.

Common wrong answers.

- `a`: Under Copy-on-Write the assignment goes to a temporary copy. Use df.loc[mask, "weight"] = 0.
- `c`: ChainedAssignmentError is a warning, not an exception. The code keeps running with df unchanged.

- `d`: The filter only selects rows. And the write never reaches df anyway.
- `b|r2`: Right answer, wrong reason. df.loc[mask, col] = value still changes df.

B3-L3 exercise 2

Hints. (1) Is s a view you can write through? (2) When does Copy-on-Write copy? (3) On the first write to shared data.

Answer.

```
1 [1, 2, 3]
```

`s` shares memory with `df` until you write to it. The write triggers a copy, so `df` keeps its values.

Common wrong answers.

- `[100, 2, 3]`: Under Copy-on-Write, s behaves as a separate object. Writing to it copies the data first, so df is unchanged.

B3-L3 exercise 3

Hints. (1) Which column names does each step need? (2) Which step depends on which rows are present? (3) Rename, filter, normalise, band.

Answer.

1. .rename(columns={"wt": "weight"})
2. .loc[lambda d: d["age"] >= 18]
3. .assign(weight=lambda d: d["weight"] / d["weight"].mean())
4. .pipe(add_age_band, bins=BINS, labels=LABELS)

Each step sees the output of the step above. Rename first so later steps use canonical names. A step that aggregates, such as a mean, must follow the filter that defines its population.

Common wrong answers.

- The normalising step reads the weight column, which does not exist until the rename.
- Normalising before the filter averages over minors too, so adult weights no longer average 1.0.

B3-L3 exercise 4

Hints. (1) Filter with .loc[lambda d: ...]. (2) Cap with Series.clip(upper=cap). (3) Join them: `df.loc[...].assign(weight=lambda d: d["weight"].clip(upper=cap))`.

Answer.

```python
import pandas as pd


def cap_weights(df: pd.DataFrame, cap: float) -> pd.DataFrame:
    """Keep adults and cap their weights at cap."""
    return df.loc[lambda d: d["age"] >= 18].assign(
        weight=lambda d: d["weight"].clip(upper=cap)
    )
```

`loc` then `assign` builds a new frame in two steps, with no temporary copies to write to.

B3-L3 exercise 5

Hints. (1) How many indexing steps does line 7 take before the =? (2) Where does the value land? (3) Use .loc in one step, or Series.where inside assign.

Answer. The bug is on line 7.

Fixed version:

```python
import pandas as pd


def zero_invalid(df: pd.DataFrame) -> pd.DataFrame:
    """Set weight to 0 where valid is False."""
    return df.assign(weight=df["weight"].where(df["valid"], 0.0))
```

`out[mask]["weight"] = 0.0` writes to a temporary copy.
`df.assign(weight=df["weight"].where(df["valid"], 0.0))` keeps weights where valid and puts 0.0 elsewhere.

Common wrong answers.

- Line 6: copy() is harmless. Under Copy-on-Write it is also cheap until a write.
- Line 8: Returning out is fine. out never received the zeros.